

FULL STACK DEVELOPMENT

- Full Stack Development refers to the practice of developing both the front-end (client-side) and back-end (server-side) portions of web applications.
- A full stack developer is proficient in working with both the front-end and back-end technologies, allowing them to build complete web applications independently or as part of a team.

Technologies used in full stack development

Front-End Technologies:

- HTML (Hypertext Markup Language): Used for structuring web pages.
- CSS (Cascading Style Sheets): Used for styling the appearance of web pages.
- JavaScript: A programming language used for adding interactivity and dynamic behavior to web pages.
- Front-end frameworks/libraries such as React.js, AngularJS, or Vue.js: These provide tools and utilities for building user interfaces and managing application state.

Back-End Technologies:

- Server-side languages like JavaScript (Node.js), Python (Django, Flask), Ruby (Ruby on Rails), Java (Spring Boot), or PHP (Laravel), C#(.Net Framework), java(Servlets)
- Databases such as MySQL, PostgreSQL, MongoDB, or Firebase for storing and managing data.
- Web servers like Apache or Nginx or IIS or Tomcat or Caddy(with built in support for https) for handling HTTP requests.

Development Tools and Environment:

- Version control systems like Git for managing code changes.
- Integrated Development Environments (IDEs) such as Visual Studio Code, Sublime Text, or Atom.
- Command-line tools for tasks like package management (npm for Node.js, pip for Python, nugget for .Net), running servers, and deployment.

Important STACKS

- MEAN Stack
- MERN Stack
- Django Stack
- LAMP
- WAMP

Why Django?

- Rapid Development:- DRY (Don't repeat Yourself)
- Saves Time and Money
- Rounded Solution
- Great Exposure
- Complete Ownership
- Greater opportunity with more learning
- Security (Prevention of threats such as SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), and clickjacking by providing {% csrf_token %} django tag to be included inside the form.

This token generates a hidden input field containing a unique CSRF token. This token is then validated by Django when the form is submitted, ensuring that the request originated from the same site and protecting against CSRF attacks.

- Batteries Included Philosophy (Model Forms)
- Built in database (Sqlite DB)

What is Django

- Django is a free and open source web application framework which offers fast and effective dynamic website development.
- It is written using python.
- It follows MVT (model, view and template architecture)

Framework

A framework is a pre-built collection of libraries, modules, and tools that provides a structured approach to developing software applications.

- Django is a web development framework.

- AngularJS is a Web Frontend development framework
- React is a Web frontend development library.

Django Evolution

1. Write a Web application from scratch.
2. Write another Web application from scratch.
3. Realize the application from step 1 shares much in common with the application from step 2.
4. Refactor the code so that application 1 shares code with application 2.
5. Repeat steps 2–4 several times.
6. Realize you've invented a framework

Django MVT

The MVT is a software design pattern which includes three important components Model, View and Template.

- The **Model** helps to handle database. It is a data access layer which handles the data.
- The **Template** is a presentation layer which handles User Interface part completely.
- The **View** is used to execute the business logic and interact with a model to carry data and renders a template.

Characteristics of Django

- Loosely Coupled – Django helps you to make each element of its stack independent of the others.
- Less code - Ensures effective development
- Not repeated- Everything should be developed in precisely one place instead of repeating it again
- Fast development- Django's offers fast and reliable application development.
- Consistent design - Django maintains a clean design and makes it easy to follow the best web development practices.

Python Virtual Environment

- A Python Virtual Environment is an isolated space where you can work on your Python projects, separately from your system-installed Python.
- You can set up your own libraries and dependencies without affecting the system Python.
- There are no limits to the number of virtual environments
- It allows you to have multiple Python environments with different versions of Python and different sets of installed packages on the same system.
- It is generally good to have one new virtual environment for every Python-based project you work on
- You can change the system python version, django version and other dependencies without affecting the project python version, django versions and dependencies.

Experiment-01

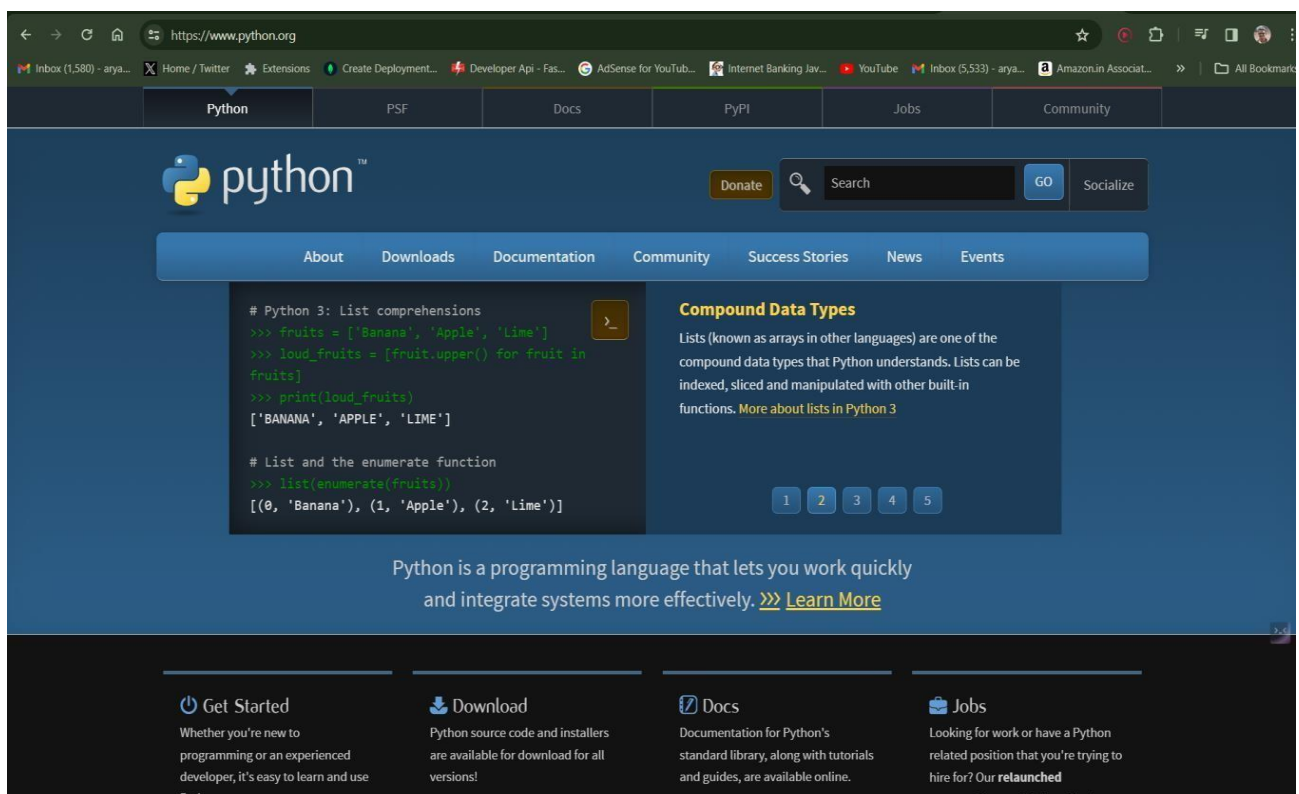
Installation of Python, Django and Visual Studio code editors can be demonstrated.

Python

- Python is a popular high-level, general-use programming language.
- Python is a programming language that enables rapid development as well as more effective system integration.
- Python has two main different versions: Python2 and Python 3. Both are really different.

Here are the steps you can follow to download Python: Steps to Download & Install Python

Visit the link <https://www.python.org> to download the latest release of Python.



Step -1: Select the Python's version to download.

Click on the download button to download the exe file of Python.

The screenshot shows the Python.org homepage. At the top, there's a navigation bar with links: Python, PSF, Docs, PyPI, Jobs, and Community. Below this is a search bar and a 'Donate' button. A secondary navigation bar includes links: About, Downloads, Documentation, Community, Success Stories, News, and Events. The main content area features a large banner with the text 'Download the latest version for Windows' and a button 'Download Python 3.12.2'. Below this, there's a paragraph: 'Looking for Python with a different OS? Python for Windows, Linux/UNIX, macOS, Other' and a link 'Want to help test development versions of Python 3.13? Prereleases, Docker images'. To the right of the text is an illustration of two parachutes carrying boxes. Below the banner is a section titled 'Active Python Releases' with a link to the 'Python Developer's Guide'. It contains a table with the following data:

Python version	Maintenance status	First released	End of support	Release schedule
3.13	prerelease	2024-10-01 (planned)	2029-10	PEP 719
3.12	bugfix	2023-10-02	2028-10	PEP 693
3.11	bugfix	2022-10-24	2027-10	PEP 664
3.10	security	2021-10-04	2026-10	PEP 640

If in case you want to download the specific version of Python. Then, you can scroll down further below to see different versions from 2 and 3 respectively. Click on download button right next to the version number you want to download.

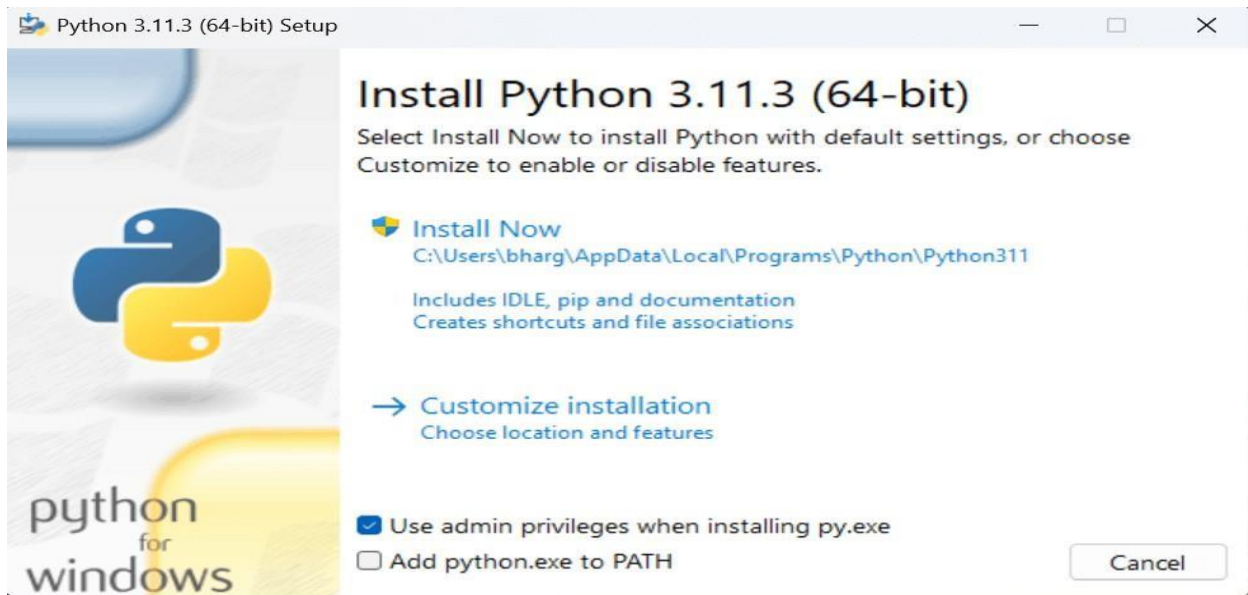
The screenshot shows the 'Looking for a specific release?' section on Python.org. It includes a link 'Python releases by version number:'. Below this is a table with the following data:

Release version	Release date	Download	Click for more
Python 3.11.9	April 2, 2024	Download	Release Notes
Python 3.10.14	March 19, 2024	Download	Release Notes
Python 3.9.19	March 19, 2024	Download	Release Notes
Python 3.8.19	March 19, 2024	Download	Release Notes
Python 3.11.8	Feb. 6, 2024	Download	Release Notes
Python 3.12.2	Feb. 6, 2024	Download	Release Notes
Python 3.12.1	Dec. 8, 2023	Download	Release Notes

Below the table is a link 'View older releases'. Further down is a 'Sponsors' section with the text 'Visionary sponsors help to host Python downloads.' and logos for Google, Meta, fastly, and Bloomberg Engineering. At the bottom, there are links: Licenses, Sources, Alternative, and History.

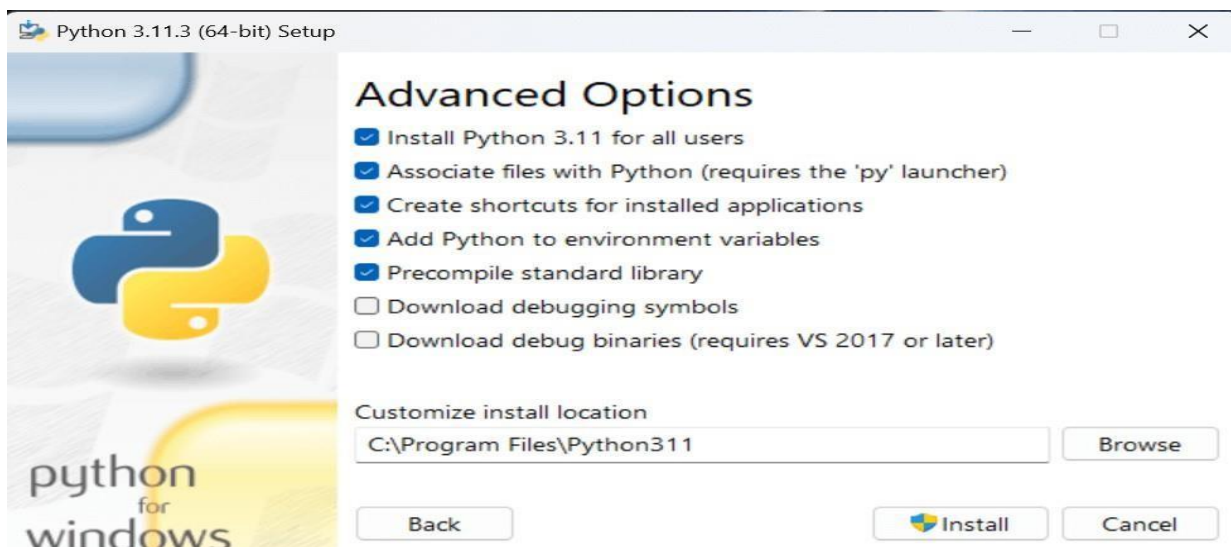
Step -2: Click on the Install Now

Double-click the executable file, which is downloaded.

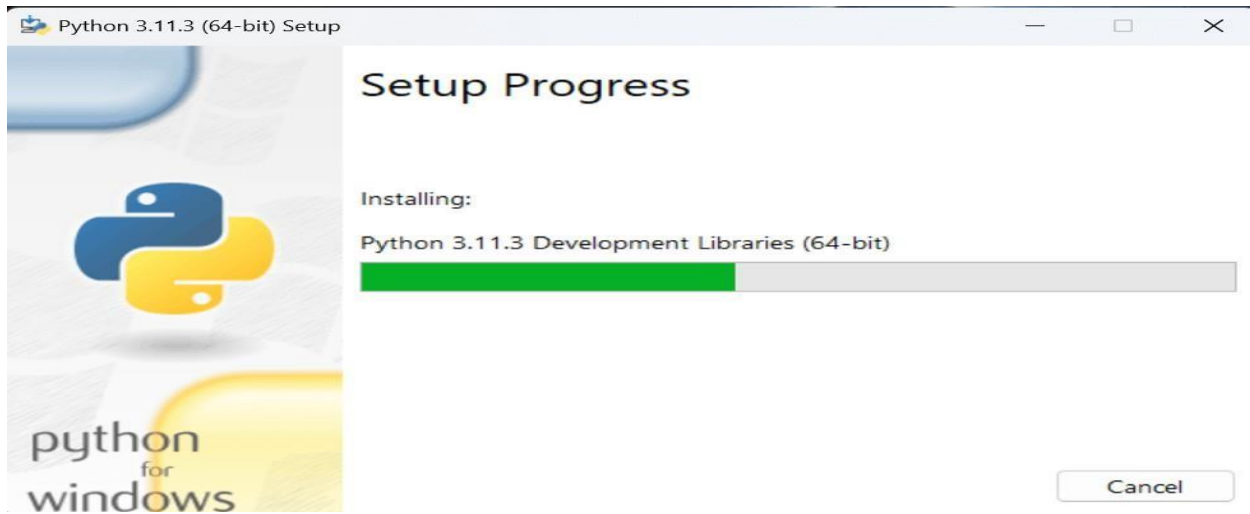


The following window will open. Click on the Add Path check box, it will set the Python path automatically.

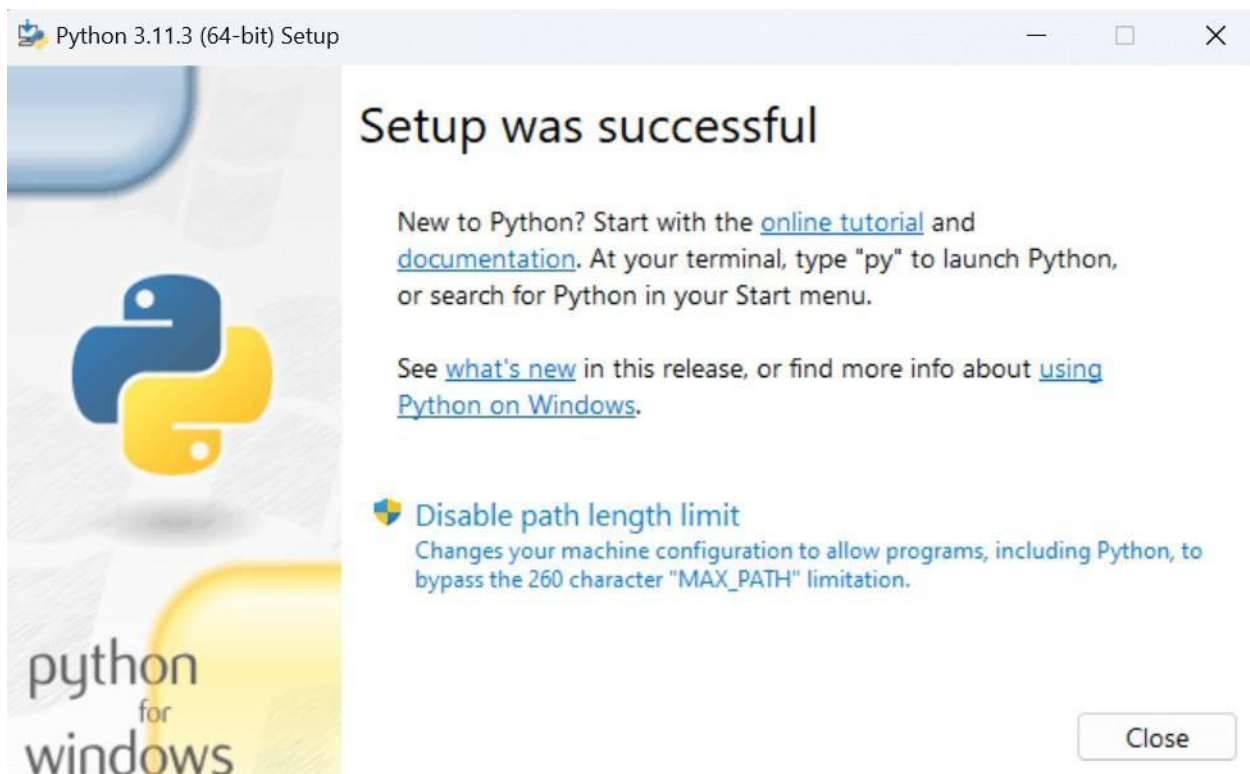
Now, Select Customize installation and proceed. We can also click on the customize installation to choose desired location and features. Other important thing is installing launcher for the all user must be checked.



Step -3 Installation in Process



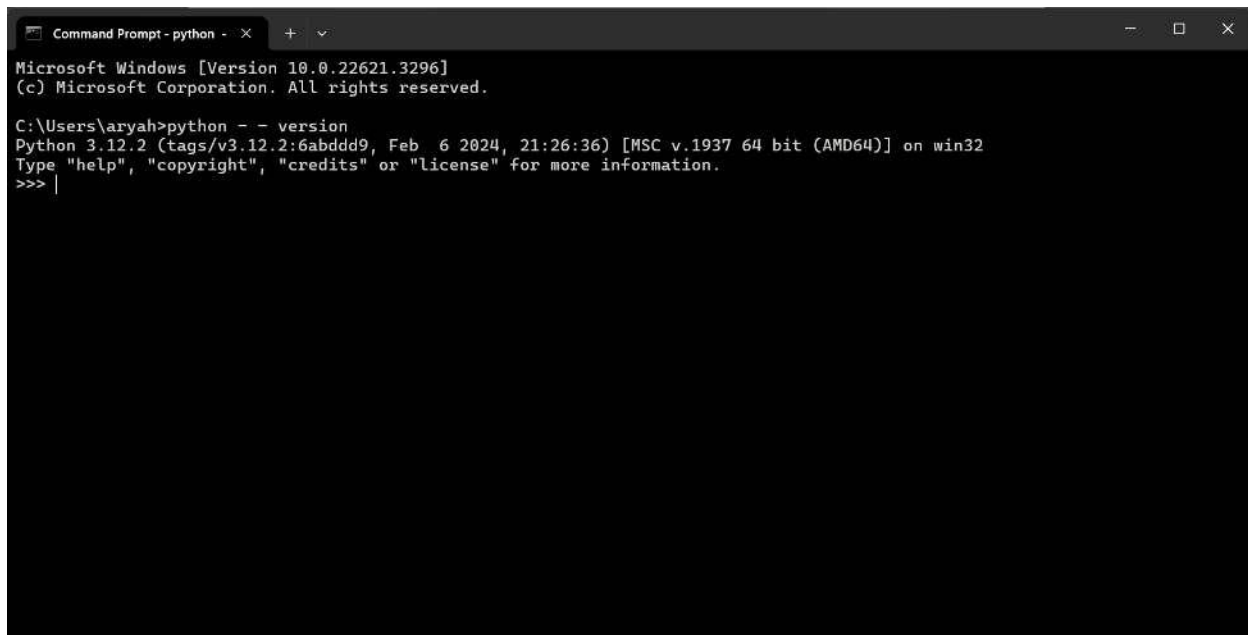
The setup is in progress. All the python libraries, packages, and other python default files will be installed in our system. Once the installation is successful, the following page will appear saying "Setup was successful".



Step -4: Verifying the Python Installation

To verify whether the python is installed or not in our system, we have to do the following.

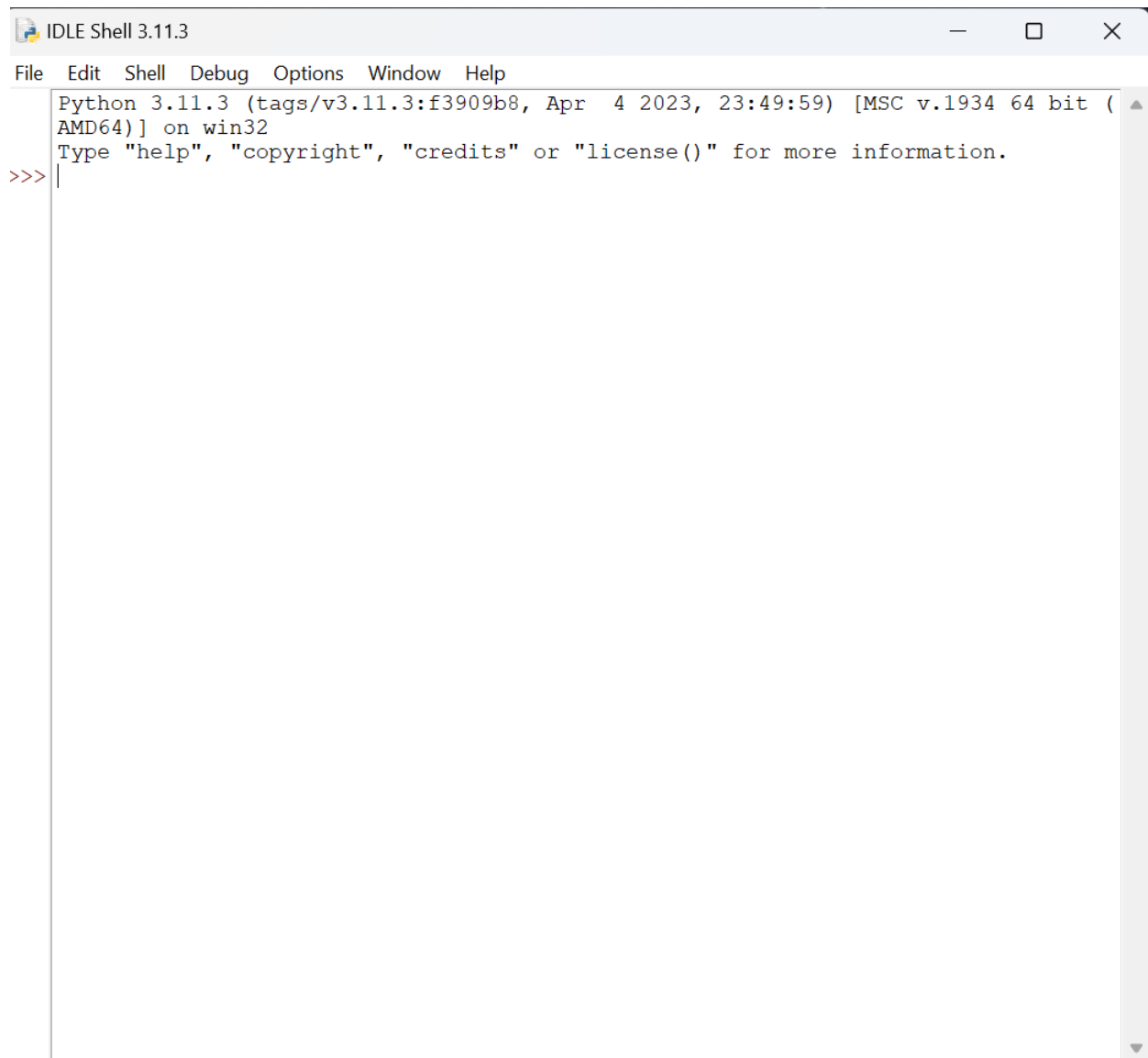
- Go to "Start" button, and search "cmd".
- Then type, "python --version".
- If python is successfully installed, then we can see the version of the python installed.
- If not installed, then it will print the error as "'python' is not recognized as an internal or external command, operable program or batch file."



```
Command Prompt - python - X + v
Microsoft Windows [Version 10.0.22621.3296]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aryah>python --version
Python 3.12.2 (tags/v3.12.2:6abddd9, Feb  6 2024, 21:26:36) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

We are ready to work with the Python.

Step-5: Opening idle

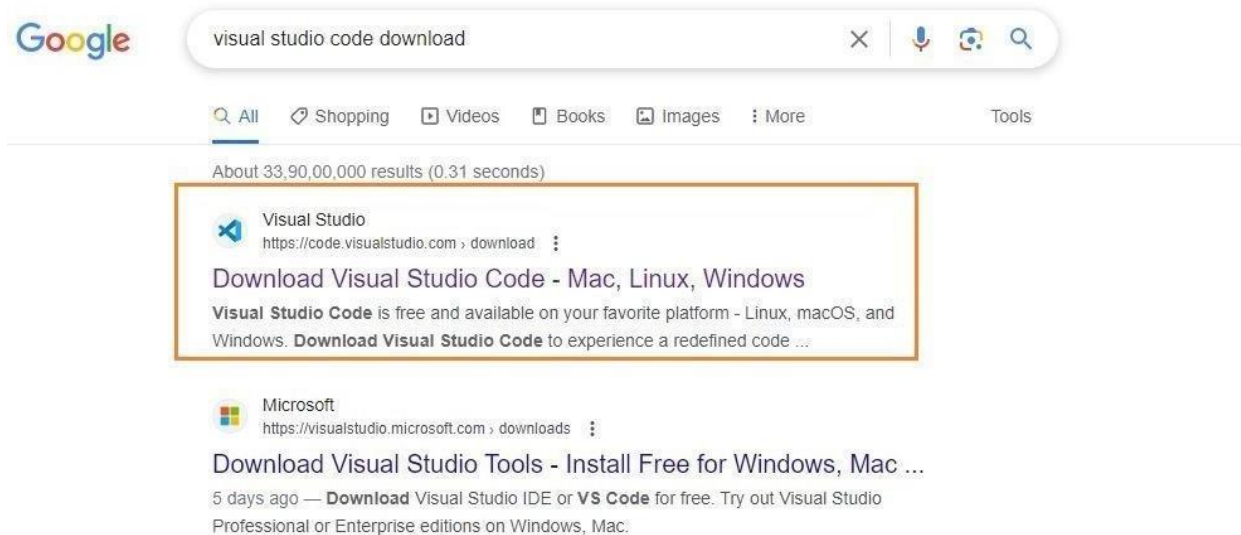
Now, to work on our first python program, we will go the interactive interpreter prompt(idle).To open this, go to "Start" and type idle. Then, click on open to start working on idle.

Here are the steps you can follow: Steps to Download & Install VS Code

Step –1: Open Google and type Visual Studio Code download in the search bar.



Step –2: Click on the below highlighted link for any OS.

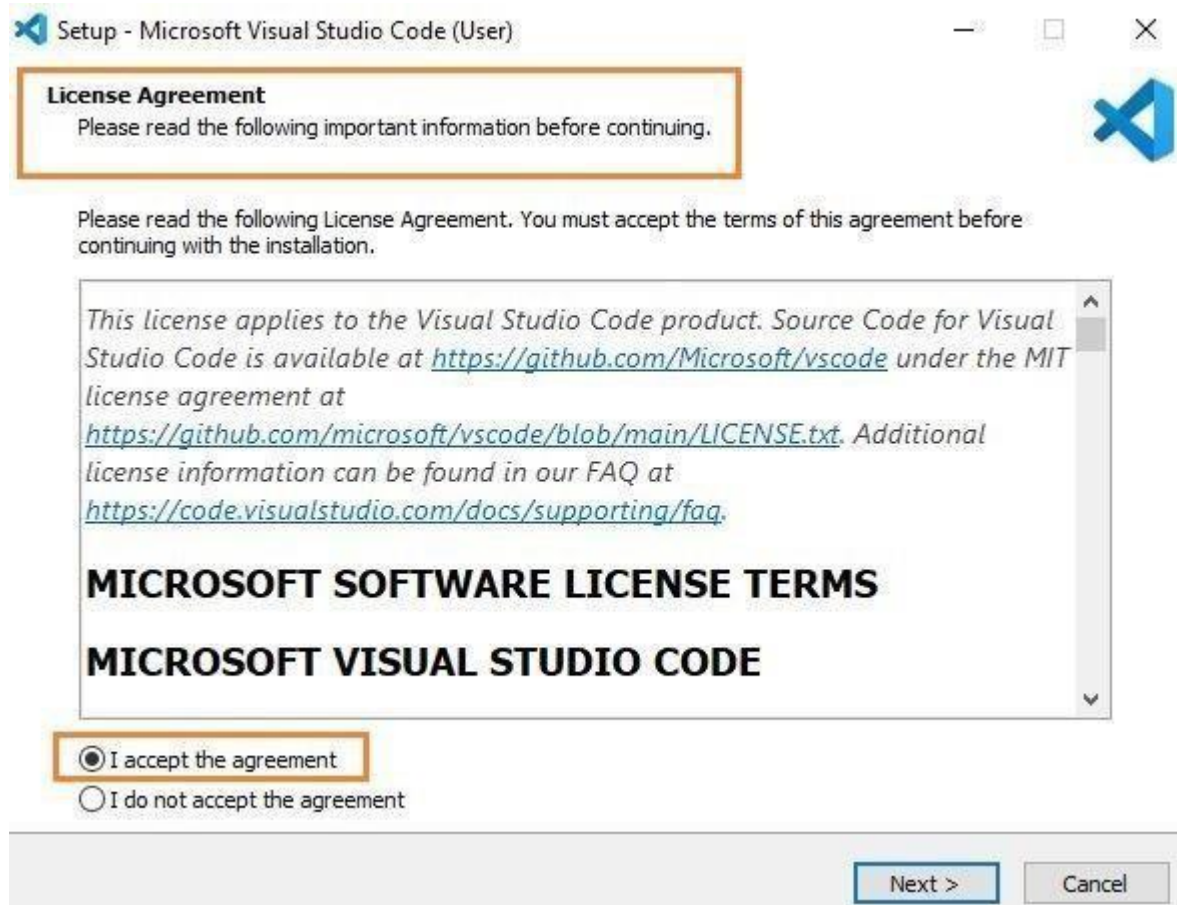


Step-3: Now, select the respective OS. In this case we are selecting Windows.



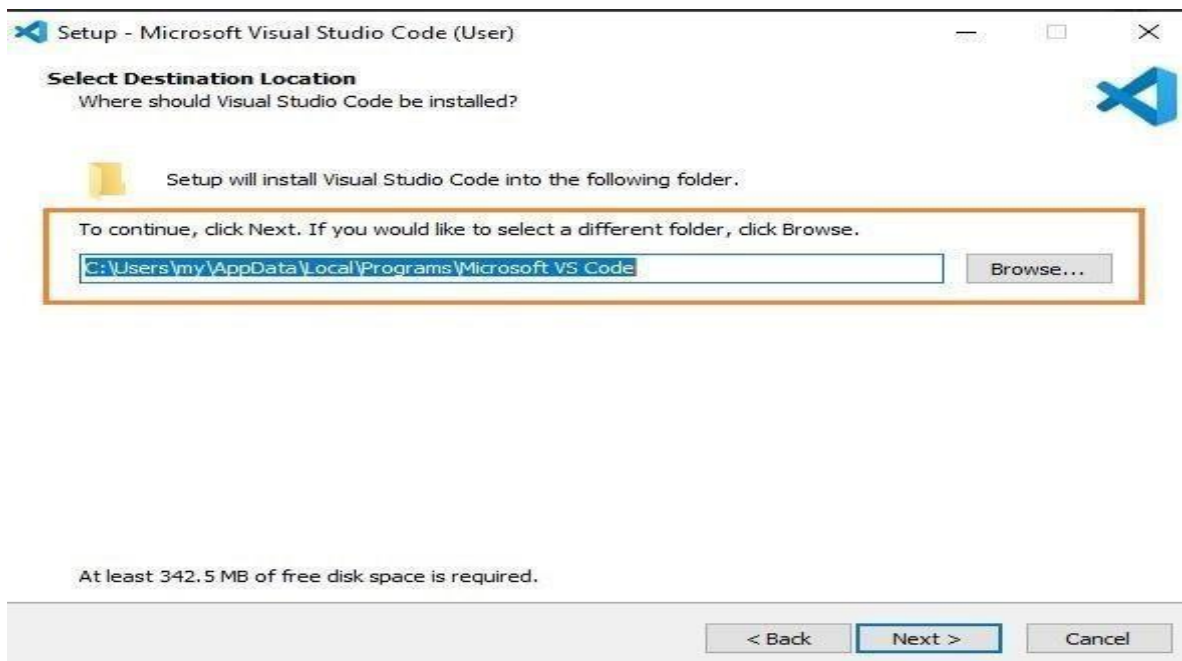
Step-4: The file will be downloaded on to your system. Open the file and then click on Install. After downloading the VS Code file, the official site will display a Thanks message for downloading the file.



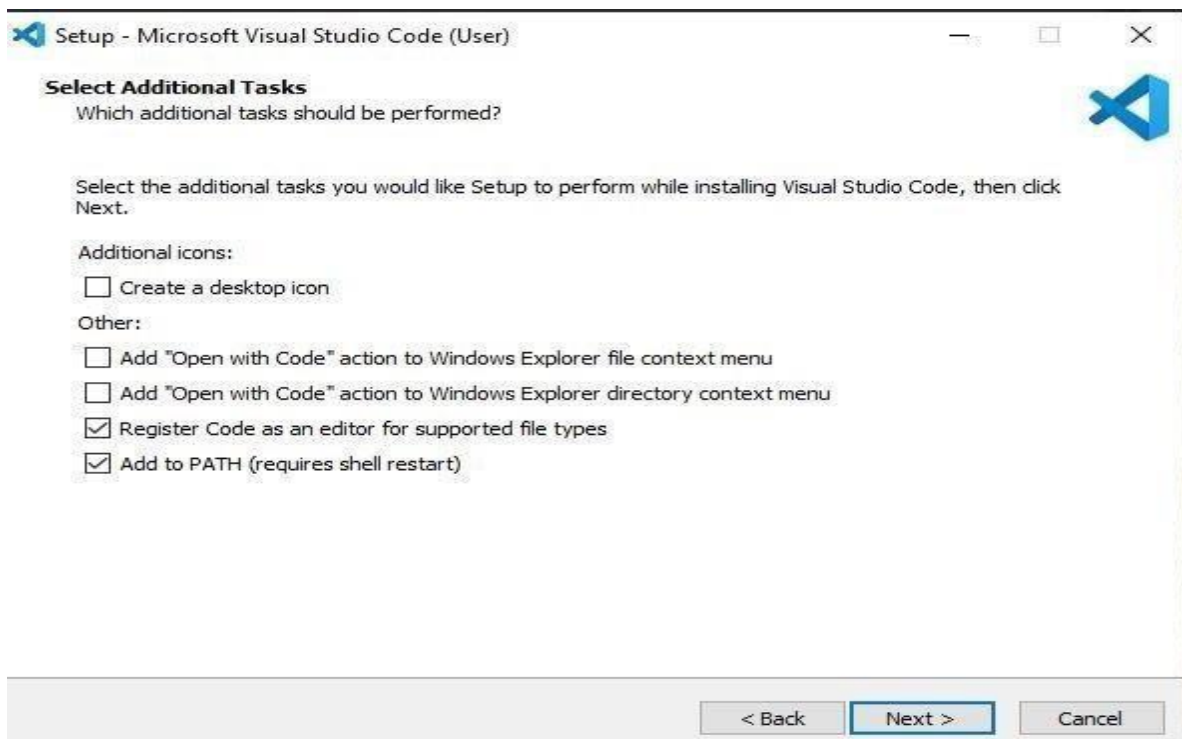
Step –5: Now accept the license agreement.

Step-6: Then it prompts for the file location, where you want to save the VSCode file.

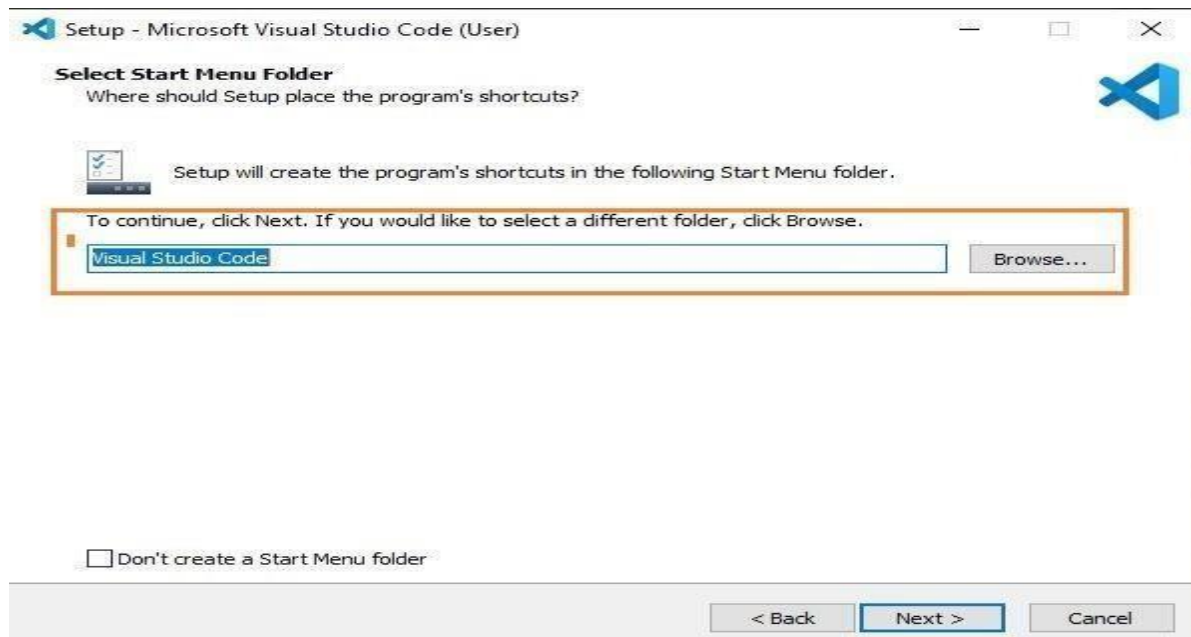
Browse the location and then click on Next.



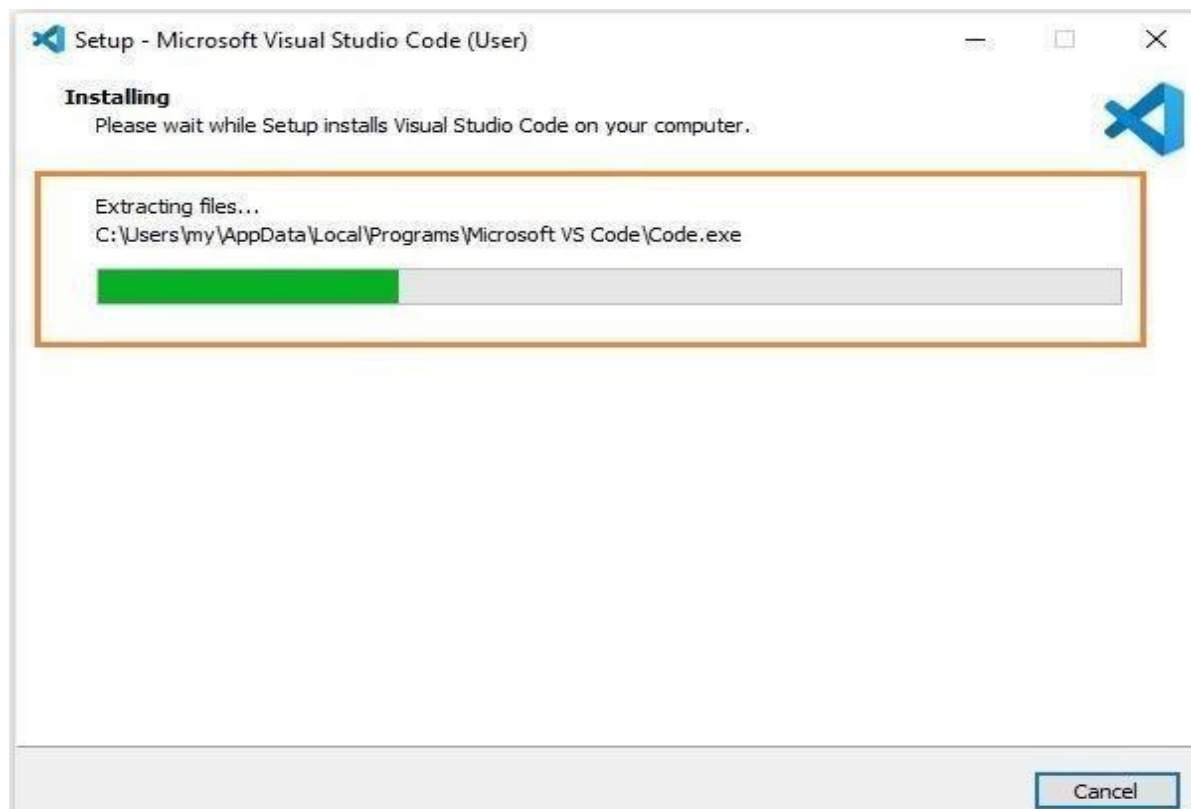
Step-7: Next ,you see the prompt for the additional task which we want the VS Code to perform. At this step, choose the default settings and then click on next.



Step–8: The next prompt is how you want the VS Code on your startup.Change according to your convenience and click on Next.



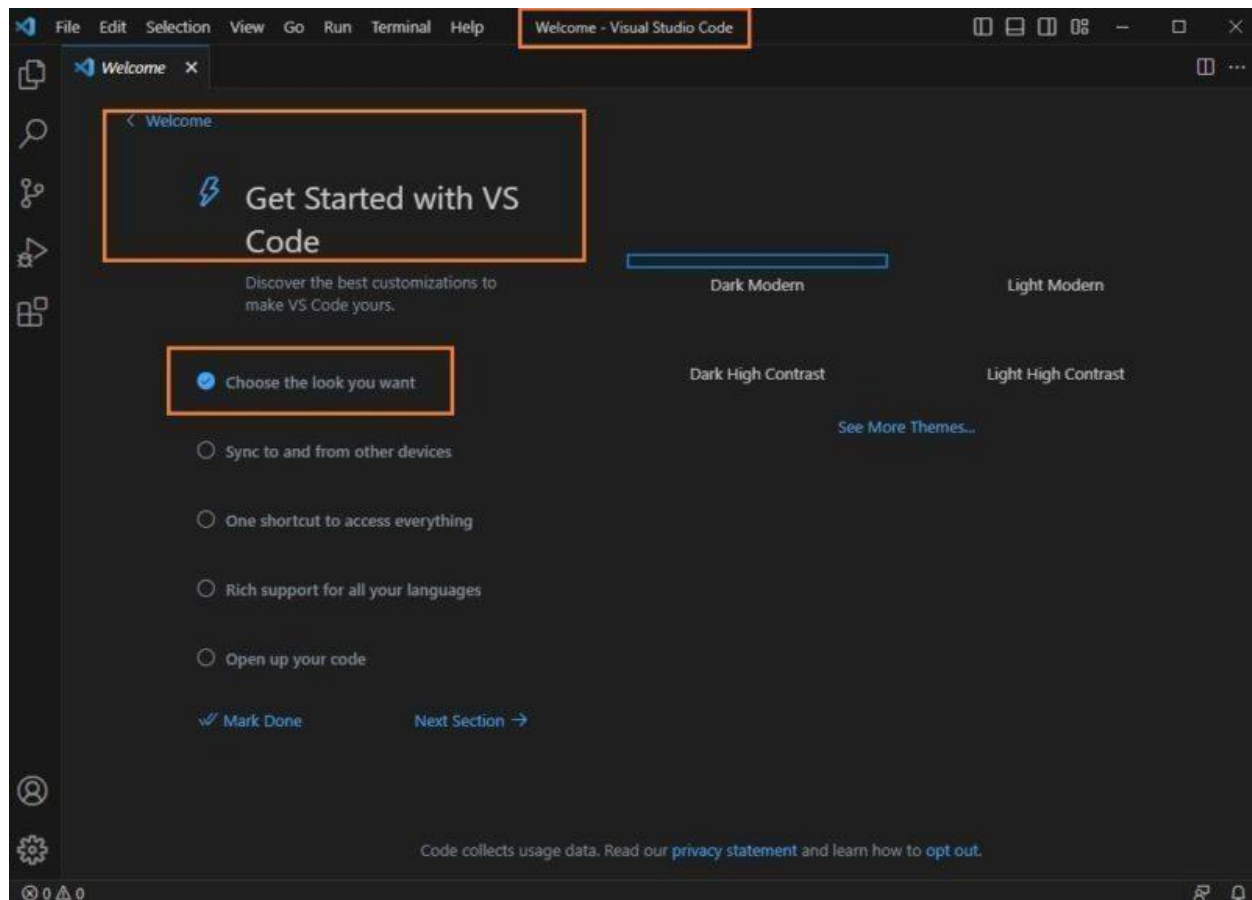
Step –9: The installation of VS Code will now begin.



Step-10: At this step, we have completed installing VS Code, click on Finish.



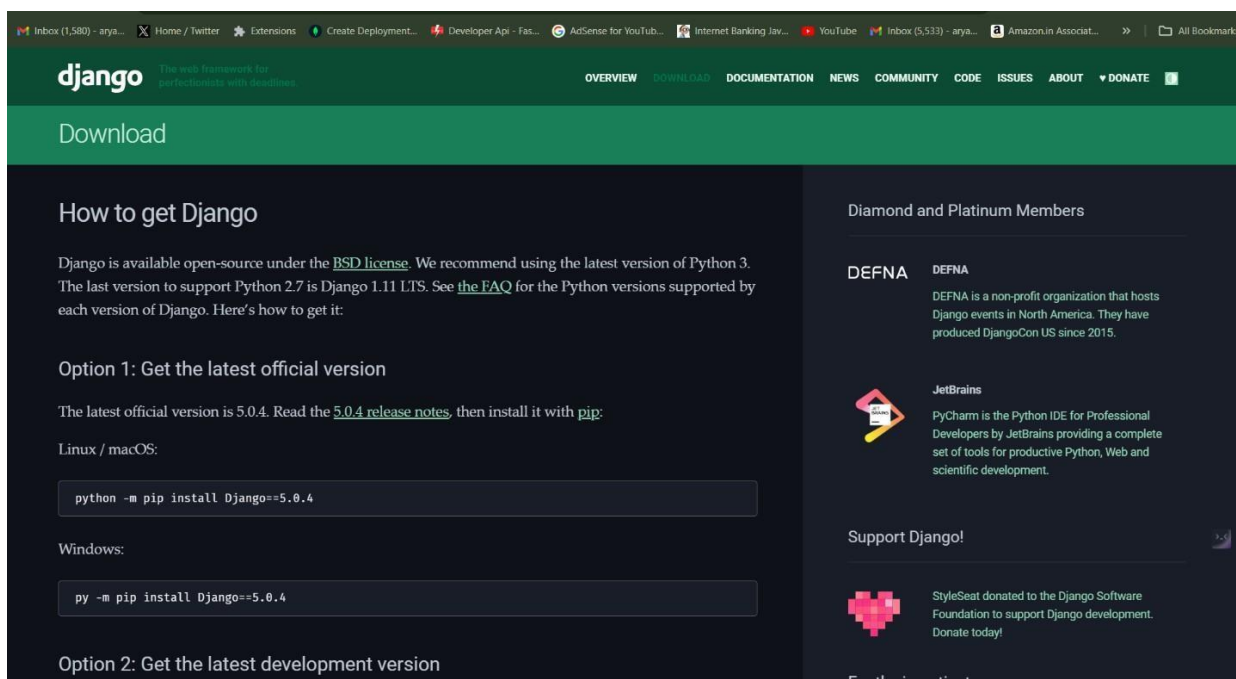
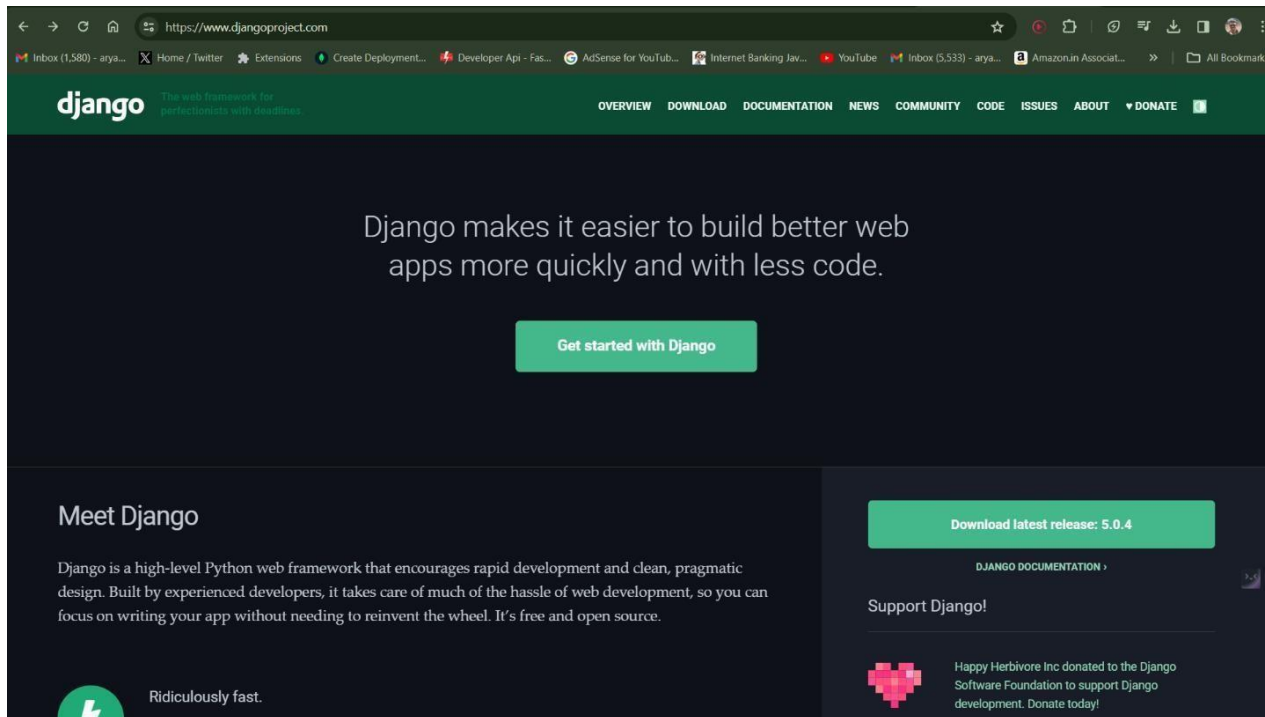
Step –11: Now that VS Code installation is successful, the page appears as below:



We can change the look as per our choice and continue working on it.

Here are the steps you can follow: Steps to Install Django

To install Django, first visit to **Django official site** (<https://www.djangoproject.com>) and download Django by clicking on the download section.

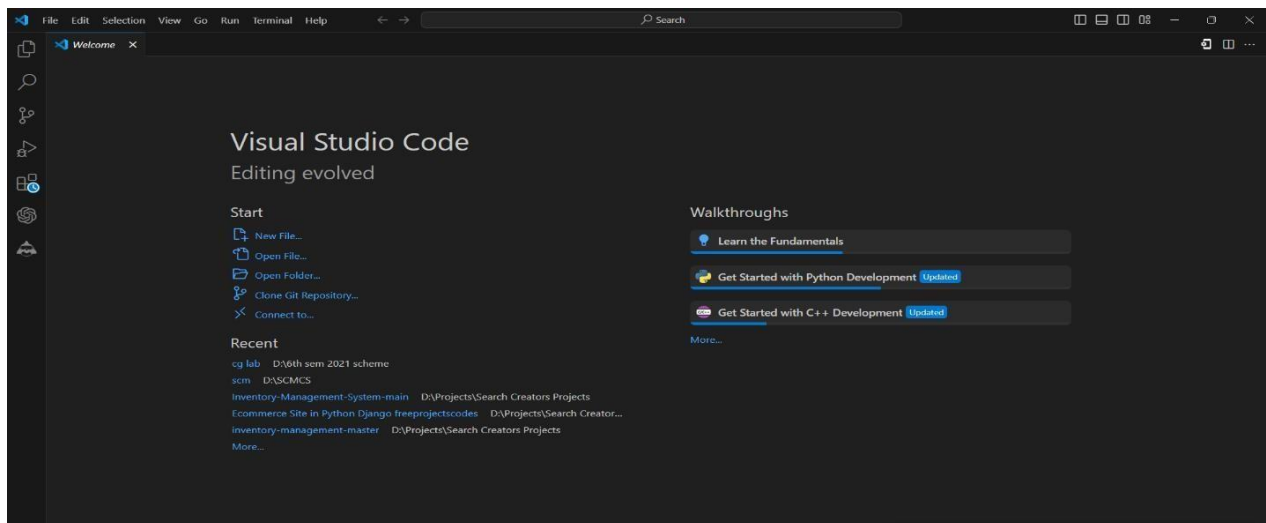


Experiment-02

Creation of virtual environment, Django project and App should be demonstrated.

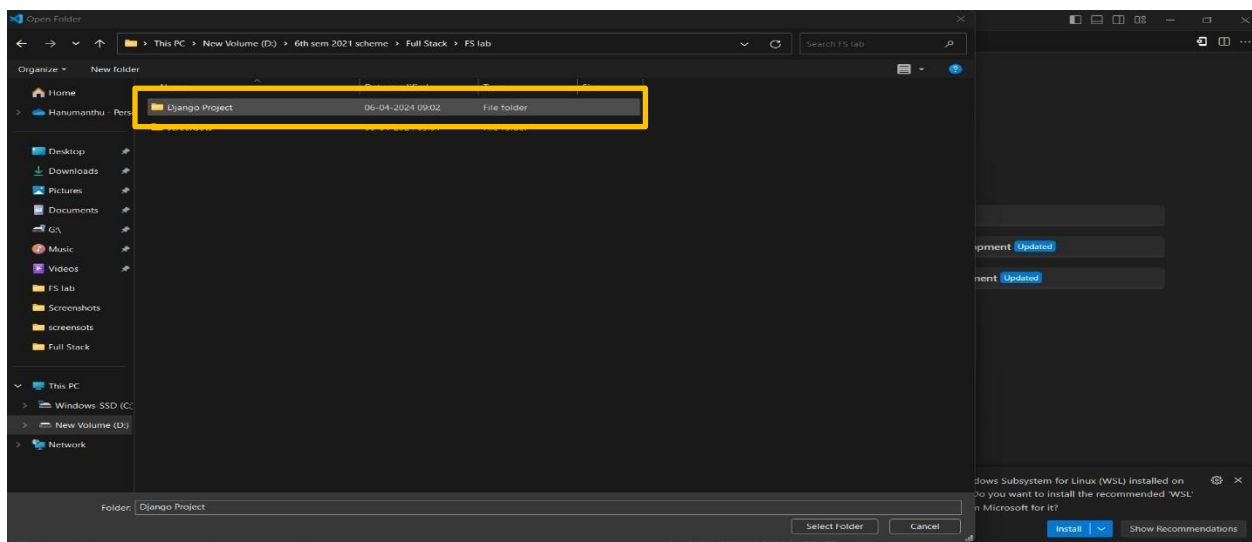
Here are the steps you can follow to create virtual environment: Steps to Create Virtual Environment, Django Project and App

Step-01: Open Visual Studio Code



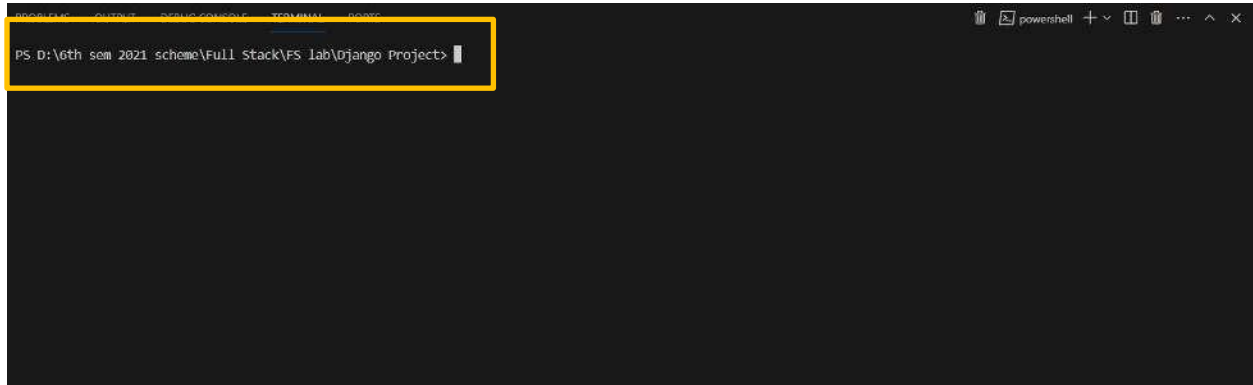
Step-02: Create a new folder for your project

In VS Code, go to **File > Open...** and create a new folder or select an existing folder where you want to create your Django project.



Step-03: Open the integrated terminal

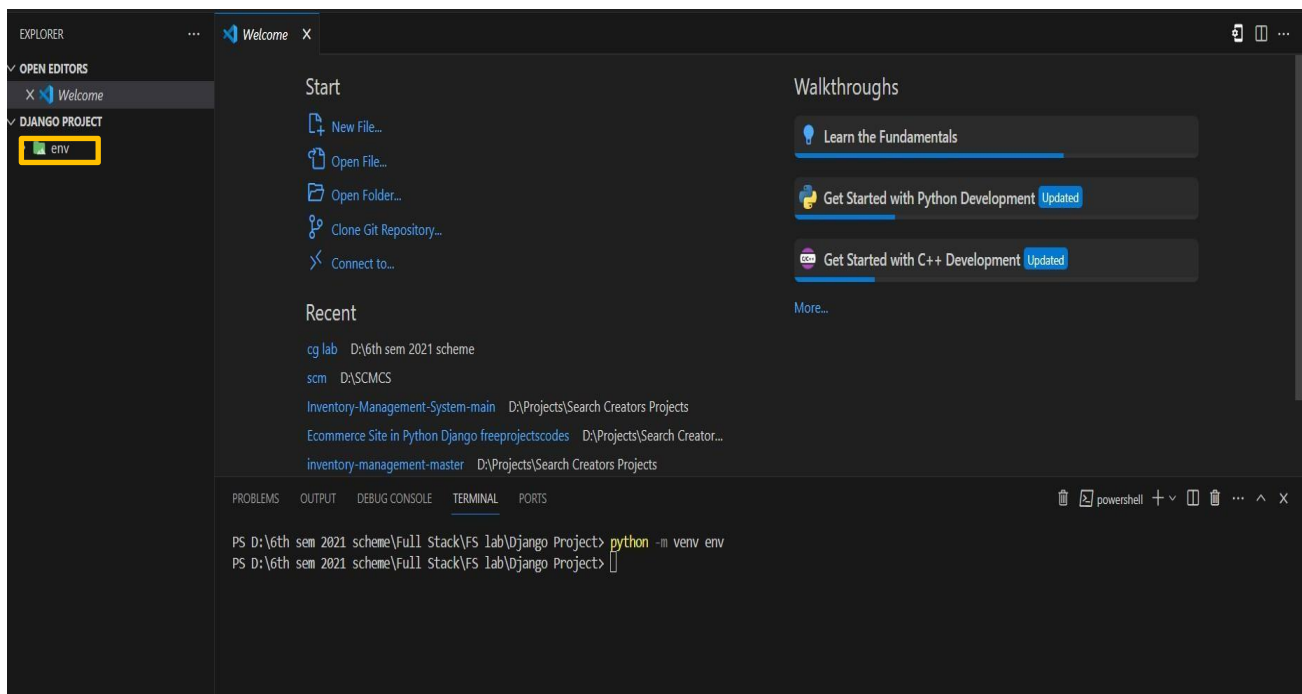
In VSCode, go to **View>Terminal** or use the keyboard shortcut Ctrl+` (Windows/Linux) or Cmd+` (macOS) to open the integrated terminal.



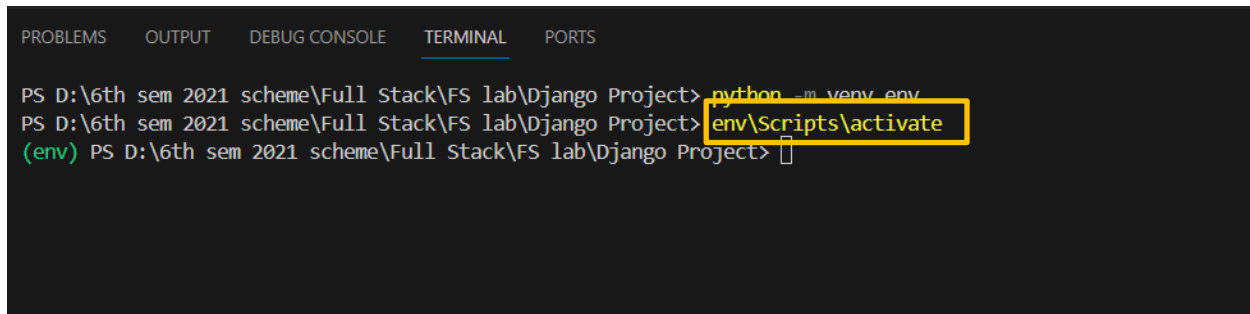
Step-03: Create a virtual environment

In the terminal, run the following command to create a new virtual environment:

python -m venv env



Step-04: Activate the virtual environment

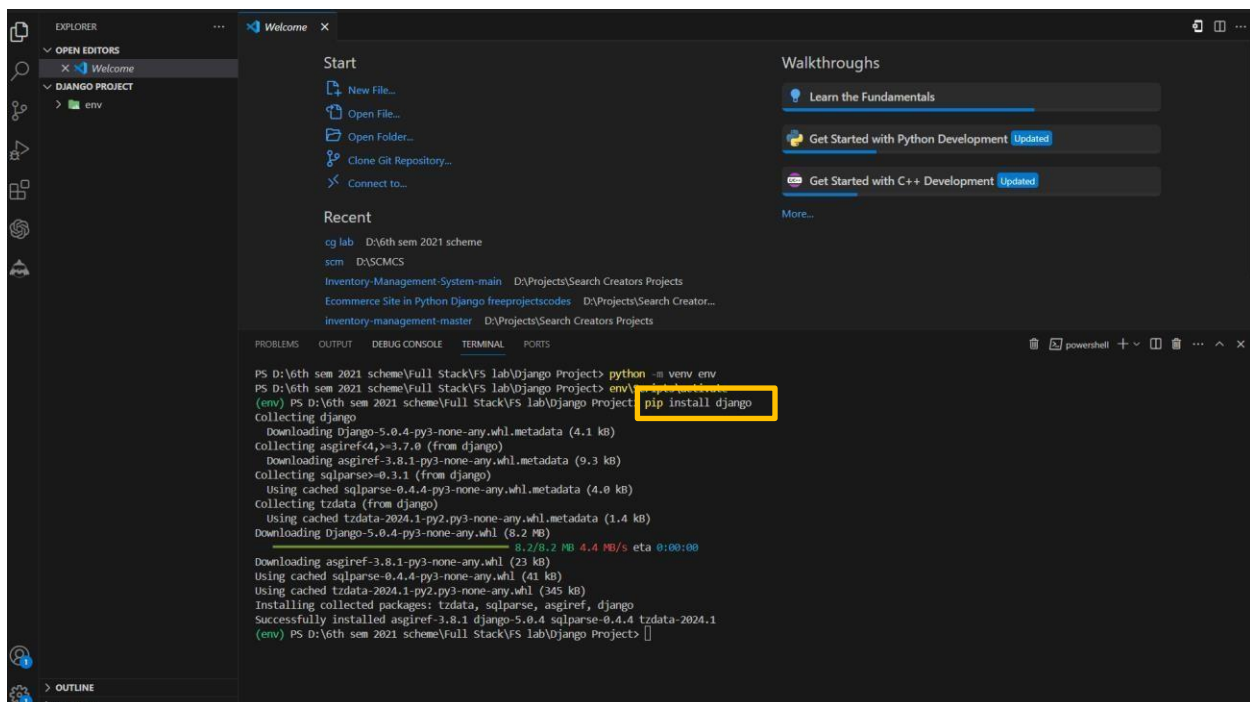


The screenshot shows a terminal window with the following commands and output:

```
PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> python -m venv env
PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> env\Scripts\activate
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project>
```

Step-05: Install Django

With the virtual environment active, install Django by running the following command in the terminal: **pip install django**



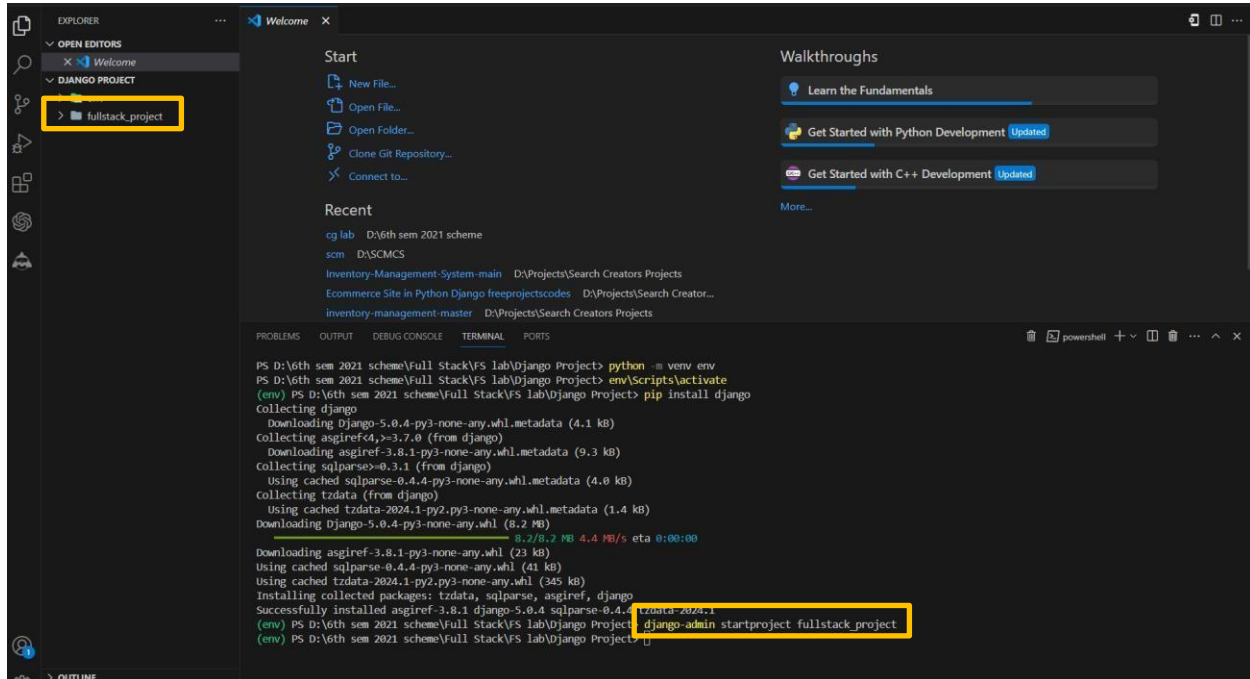
The screenshot shows a terminal window with the following commands and output:

```
PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> python -m venv env
PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> env\Scripts\activate
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project> pip install django
Collecting django
  Downloading Django-5.0.4-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<4,=>3.7.0 (from django)
  Downloading asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse==0.3.1 (from django)
  Using cached sqlparse-0.4.4-py3-none-any.whl.metadata (4.0 kB)
Collecting tzdata (from django)
  Using cached tzdata-2024.1-py2.py3-none-any.whl.metadata (1.4 kB)
  Downloading Django-5.0.4-py3-none-any.whl (8.2 MB)
    8.2/8.2 MB 4.4 MB/s eta 0:00:00
  Downloading asgiref-3.8.1-py3-none-any.whl (23 kB)
  Using cached sqlparse-0.4.4-py3-none-any.whl (41 kB)
  Using cached tzdata-2024.1-py2.py3-none-any.whl (345 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.8.1 django-5.0.4 sqlparse-0.4.4 tzdata-2024.1
(env) PS D:\6th sem 2021 scheme\Full Stack\F5 lab\Django Project>
```

Step-06: Create a new Django project

Run the following command to create a new Django project:

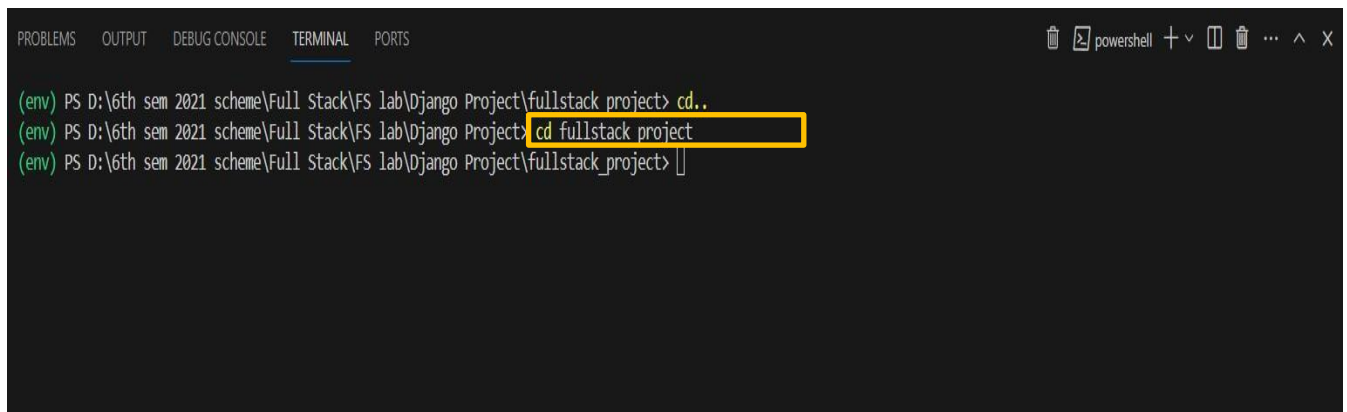
django-admin startproject myproject [change Project name as You Desired]



Step-07: Create a new Django app

In the VS Code terminal, navigate to your project directory:

cd myproject [change Project name as You Desired]



Create a new Django app by running the following command

Python manage.py startapp myapp

[Replace myapp with the name you want to give to your app.]

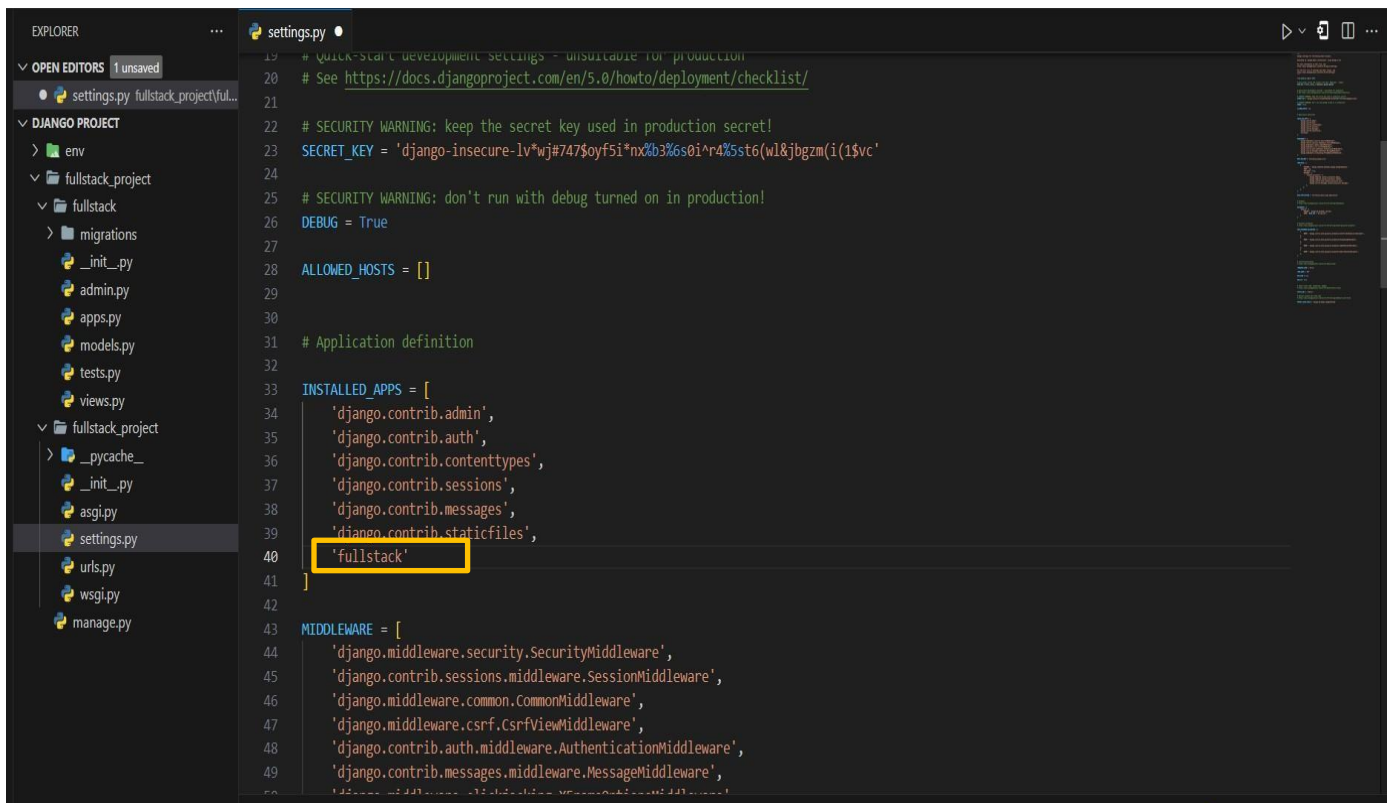


```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> cd ..  
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project> cd fullstack_project  
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py startapp fullstack  
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> |
```

Step-08: Add the app to the INSTALLED_APPS list

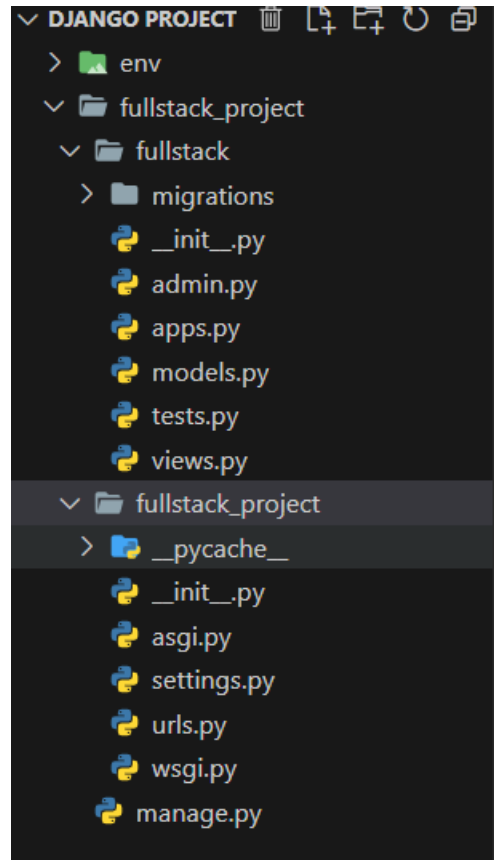
In the VSCode file explorer, locate the settings.py file(usually located in the myproject directory) and open it.

Locate the **INSTALLED_APPS** list and add the name of your new app to the list



```
19 # Quick-start development settings - unsuitable for production  
20 # See https://docs.djangoproject.com/en/5.0/howto/deployment/checklist/  
21  
22 # SECURITY WARNING: keep the secret key used in production secret!  
23 SECRET_KEY = 'django-insecure-lv*wj#747$oyf5i*n%b3%6s0i*r4%5st6(wl&jbgzm(i(1$vc'  
24  
25 # SECURITY WARNING: don't run with debug turned on in production!  
26 DEBUG = True  
27  
28 ALLOWED_HOSTS = []  
29  
30 # Application definition  
31  
32 INSTALLED_APPS = [  
33     'django.contrib.admin',  
34     'django.contrib.auth',  
35     'django.contrib.contenttypes',  
36     'django.contrib.sessions',  
37     'django.contrib.messages',  
38     'django.contrib.staticfiles',  
39     'fullstack'  
40 ]  
41  
42 MIDDLEWARE = [  
43     'django.middleware.security.SecurityMiddleware',  
44     'django.contrib.sessions.middleware.SessionMiddleware',  
45     'django.middleware.common.CommonMiddleware',  
46     'django.middleware.csrf.CsrfViewMiddleware',  
47     'django.contrib.auth.middleware.AuthenticationMiddleware',  
48     'django.contrib.messages.middleware.MessageMiddleware',  
49     'django.middleware.clickjacking.XFrameOptionsMiddleware',  
50 ]
```

Here is the Django project structure that you wanted to create



After creating your Django app, the next step is to **create database migrations** for your app's models (if you have any defined). Here's how you can do that using the **python manage.py makemigrations** command in Visual Studio Code (VS Code)

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py makemigrations
No changes detected
```

Step-08: Apply the migrations

Once you've reviewed the migration files and are ready to apply the changes to your database, run the following command in the terminal:

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

Step-09: Run Your Project

Here's how you can run the Django development server using the **python manage.py runserver** Command in Visual Studio Code(VSCode)

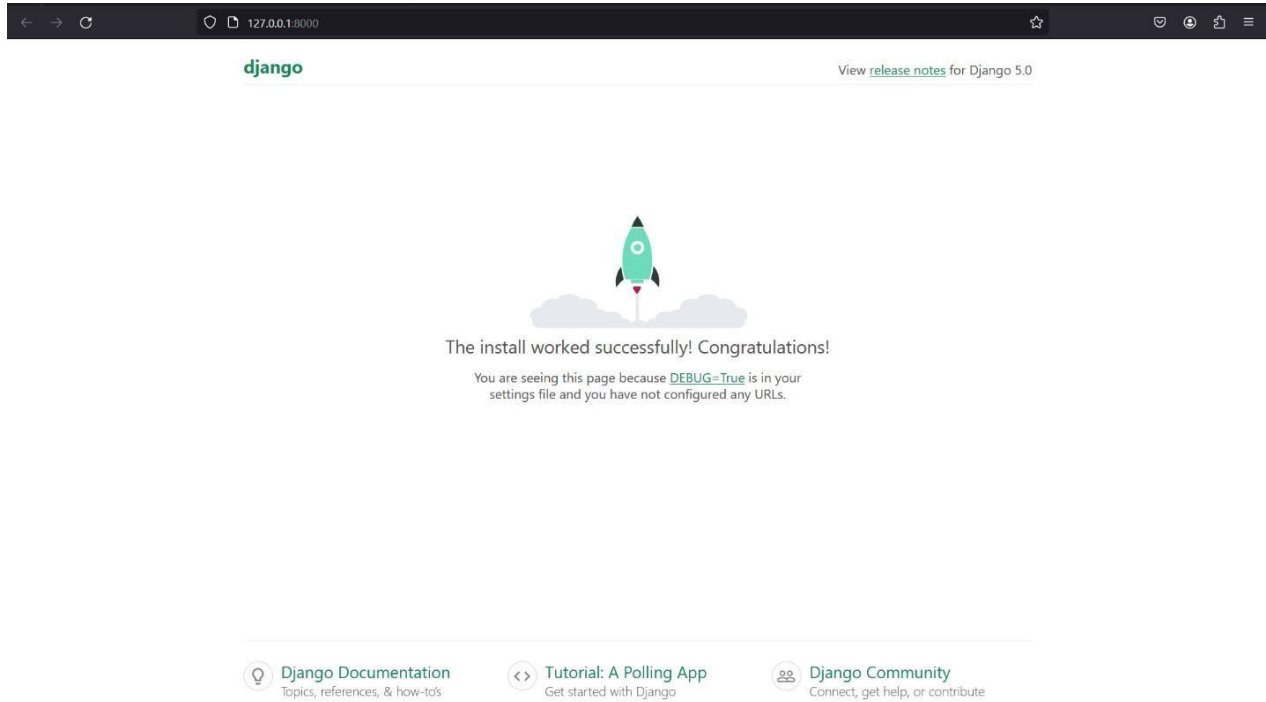
```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 11:23:06
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Check the server output

The terminal will display the URL where the development server is running, typically <http://127.0.0.1:8000/>. It will also show any startup logs or warnings, if any. Open the development server in your browser Copy the URL from the terminal output (e.g., <http://127.0.0.1:8000/>) and paste it into your web browser's address bar.

Here Displays the Development Server After Running Server



Experiment-03

Develop a Django app that displays current date and time in server.

Step-01: Create project

```
django-admin startproject datetime_project
```

Step-02: Change directory:

```
cd datetime_project
```

Create App:

```
python manage.py startapp datetime_app
```

Step-03: Create view.py in your app (datetime_app)

```
#views.py

from django.shortcuts import render

from django.utils import timezone

def current_datetime(request):

    now = timezone.now()

    context = {'datetime': now}

    return render(request, 'datetime_app/current_datetime.html', context)
```

Step-04: Create a template

- In your app's directory (e.g., **datetime_app**), create a new folder named 'templates'.
- Inside the templates folder, create another folder with the same name as your app (e.g., **datetime_app**).
- Inside the **datetime_app** folder, create a new file named **current_datetime.html**.
- Open **current_datetime.html** and add the following code to display the current date and time:

```
<!DOCTYPE html>
<html>
<head>
    <title>
        Current date and time
    </title>
</head>
<body>
    <h1>Current Date and Time:</h1>
    {{datetime}}
</body>
</html>
```

Step-05: Map the view function to a URL

- Create and open the urls.py file in your Django app's directory (e.g., datetime_app/urls.py).
- Import the view function at the top of the file
- Add a new URL pattern to the urlpatterns list

```
#urls.py

from django.urls import path

from . import views

urlpatterns = [

    path(' ',views.current_datetime, name='current_datetime'),

]
```

Step-06: Add the app to INSTALLED_APPS

- Open the settings.py file in your project's directory (e.g., datetime_project/settings.py).
- Locate the **INSTALLED_APPS** list and add the name of your new app to the list:

Step-07: Include the app's URLs in the project's URL patterns

- Open the **urls.py** file in your project's directory (e.g., **datetime_project/urls.py**).
- **Include the following code:**

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [

    path('admin/', admin.site.urls),

    path('',include('datetime_app.urls')),

]
```

- Import the include function from **django.urls** and the path function from **django.urls**:

from django.urls import include, path

- Add a new URL pattern to the urlpatterns list

path ('', include('datetime_app.urls')),

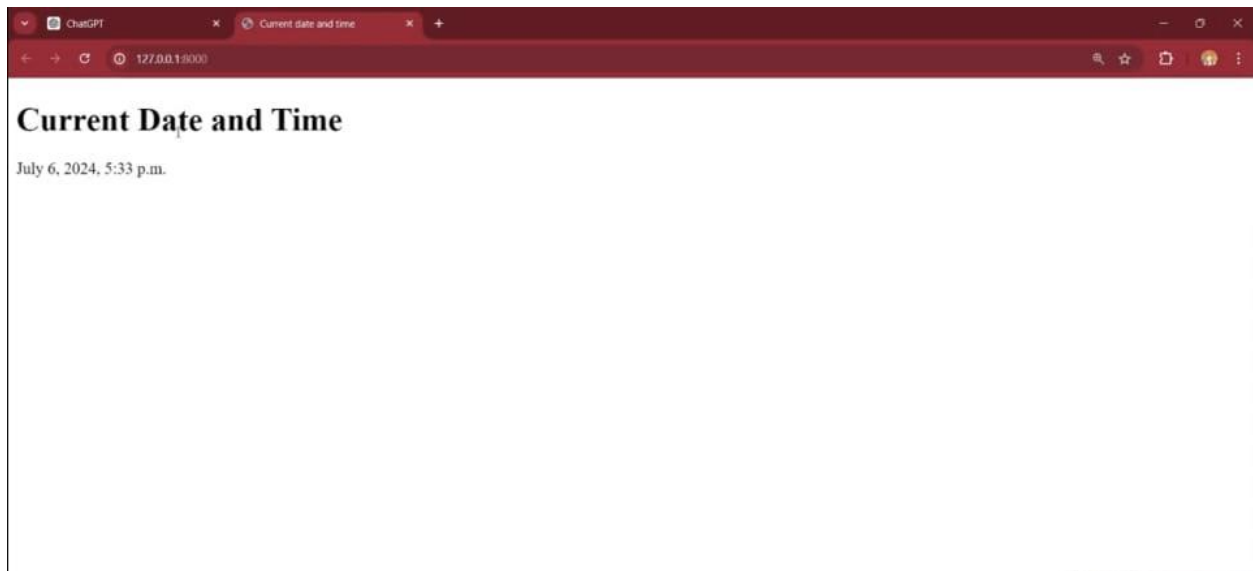
- This includes the URL patterns from your app's urls.py file.

Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.

OUTPUT:

Experiment-04

Develop a Django app that displays date and time four hours ahead and four hours before as an offset of current date and time in server.

Step-01: Create project

```
django-admin startproject fourdatetime_project
```

Step-02: Change directory:

```
cd fourdatetime_project
```

Create App:

```
python manage.py startapp fourdatetime_app
```

Step-03: Create view.py in your app (fourdatetime_app/views.py)

```
#views.py
```

```
from django.shortcuts import render
```

```
import datetime
```

```
def datetime_offsets(request):
```

```
    now = datetime.datetime.now()
```

```
    context = {
```

```
        'current_datetime': now,
```

```
        'four_hours_ahead': now + datetime.timedelta(hours=4),
```

```
        'four_hours_before': now - datetime.timedelta(hours=4),
```

```
    }
```

```
    return render(request, 'fourdatetime_app/datetime_offsets.html', context)
```

- This view function gets the current date and time using `datetime.datetime.now()`, calculates the date and time four hours ahead and four hours before using `datetime.timedelta`, and then passes all three values to the template as context variables.

Step-04: Create a new template

- In your app's directory (e.g., `fourdatetime_app`), create a new folder named `templates` (if it doesn't already exist).
- Inside the `templates` folder, create another folder with the same name as your app (e.g., `fourdatetime_app`).
- Inside the `fourdatetime_app` folder, create a new file named `datetime_offsets.html`.
- Open `datetime_offsets.html` and add the following code to display the current date and time, along with the offsets:

```
<!DOCTYPE html>
<html>
<head>
  <title>
    Current date and time
  </title>
</head>
<body>
  <h1>Current Date and Time on the Server</h1>
  <p>{{ current_datetime }}</p>
  <h2>Four Hours Ahead</h2>
  <p>{{ four_hours_ahead }}</p>
  <h2>Four Hours Before</h2>
  <p>{{ four_hours_before }}</p>
</body>
</html>
```

Step-05: Map the view function to a URL

- Create and open the `urls.py` file in your Django app's directory (e.g., `fourdatetime_app/urls.py`).
- Import the view function at the top of the file
- Add a new URL pattern to the `urlpatterns` list.

```
from django.urls import path

from . import views

urlpatterns = [

    path('datetime-offsets/', views.datetime_offsets, name='datetime_offsets'),

]
```

Step-06: Add the app to INSTALLED_APPS

- Open the settings.py file in your project's directory (e.g., fourdatetime_project/settings.py).
- Locate the INSTALLED_APPS list and add the name of your new app to the list.

Step-07: Include the app's URLs in the project's URL patterns

- Open the urls.py file in your project's directory (e.g., fourdatetime_project/urls.py).
- Import the include function from **django.urls** and the path function from **django.urls**:

```
from django.urls import include, path
```

- Add a new URL pattern to the urlpatterns list

```
path('', include('fourdatetime_app.urls')),
```

- This includes the URL patterns from your app's urls.py file

- Include the following code:

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('fourdatetime_app.urls')),

]
```

Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **<http://127.0.0.1:8000/datetime-offsets/>**

OUTPUT:

Current Date and Time on the Server

April 6, 2024, 10:15 p.m.

Four Hours Ahead

April 7, 2024, 2:15 a.m.

Four Hours Before

April 6, 2024, 6:15 p.m.

Experiment-05

Develop a simple Django app that displays an unordered list of fruits and ordered list of selected students for an event.

Step-01: Create project

```
django-admin startproject listfruit_project
```

Step-02: Change directory:

```
cd listfruit_project
```

Create App:

```
python manage.py startapp listfruit_app
```

Step-03: Create view.py in your app (listfruit_app//views.py)

```
from django.shortcuts import render
def fruit_student(request):
    fruitList=['Mango','Kiwi','Banana','Apple','Grapes']
    studentList=['Rama','Chetan','Kumar','Harish','Geetha']
    context = {
        'fruitList':fruitList,
        'studentList':sorted(studentList),
    }
    return render(request, 'listfruit_app/fruits_student.html', context)
```

- Here, we define a view function ‘fruit_student’ that creates two lists: fruitList and studentList. These lists are then passed to the template as a context dictionary.

Step-04: Create a template

- In your app's directory (e.g., **listfruit_app**), create a new folder named 'templates'.
- Inside the templates folder, create another folder with the same name as your app (e.g., **listfruit_app**).
- Inside the **listfruit_app** folder, create a new file named **fruits_student.html**.
- Open **fruits_student.html** and add the following code:

fruits_student.html

```
<!DOCTYPE html>
<html>
<head>
  <style>
    #a1 {background-color: lightblue;color:brown}
    #a2 {background-color:blue;color:yellow}
  </style>
  <title>
    Unordered Fruits and Ordered Students
  </title>
</head>
<body>
  <h1 id="a1">Unordered List of Fruits</h1>
  <ul>
    {% for fruit in fruitList %}
      <li>{{fruit}}</li>
    {% endfor %}
  </ul>
  <h1 id="a2">Ordered List of Students Selected for an Event</h1>
  <ol>
    {% for student in studentList %}
      <li>{{student}}</li>
    {% endfor %}
  </ol>
</body>
</html>
```

Step-05: Map the view function to a URL

- Create and open the `urls.py` file in your Django app's directory (e.g., `listfruit_app/urls.py`).
- Import the view function at the top of the file
- Add a new URL pattern to the `urlpatterns` list

```
from django.urls import path
from . import views
urlpatterns = [
    path('fruits/', views.fruit_student, name='fruit_student '),
]
```

Step-06: Add the app to INSTALLED_APPS

- Open the settings.py file in your project's directory (e.g., listfruit_project/settings.py).
- Locate the **INSTALLED_APPS** list and add the name of your new app to the list:

Step-07: Include the app's URLs in the project's URL patterns

- Open the urls.py file in your project's directory (e.g., listfruit_project/urls.py).
- Import the include function from **django.urls** and the path function from **django.urls**:

```
from django.urls import include, path
```

- Add a new URL pattern to the urlpatterns list

```
path ('', include('listfruit_app.urls')),
```

- This includes the URL patterns from your app's urls.py file.
- Include the following code:

```
from django.contrib import admin
```

```
from django.urls import include, path
```

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

```
    path('',include('listfruit_app.urls')),
```

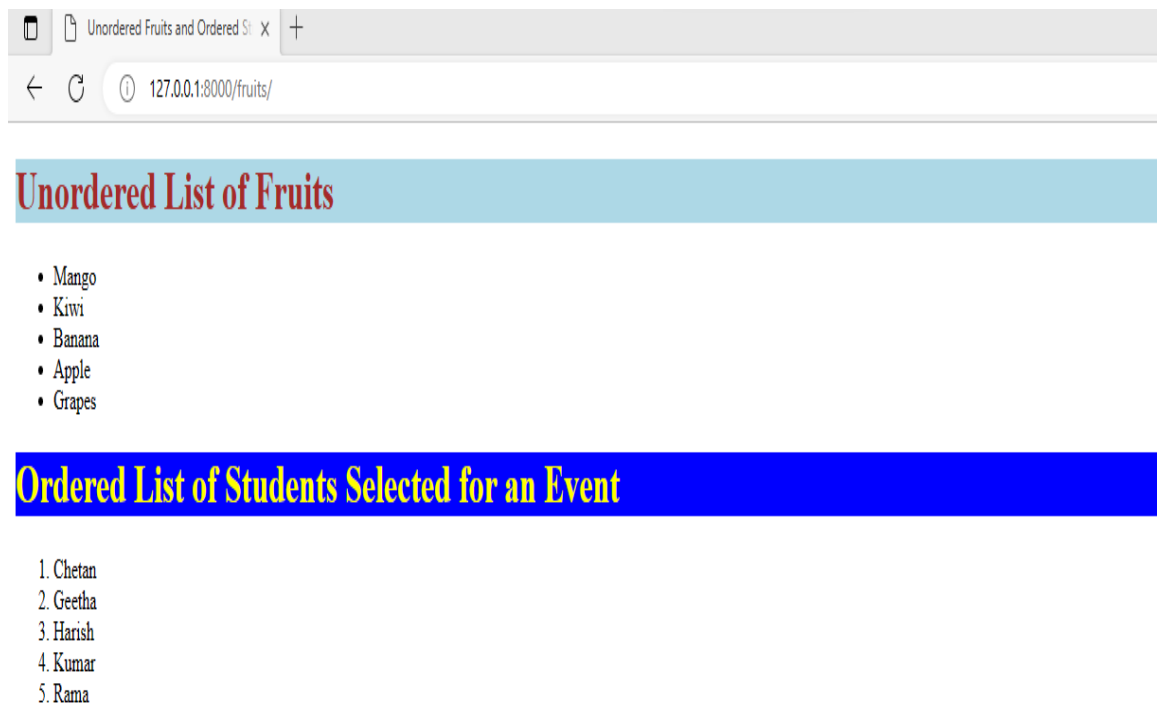
```
]
```

Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit <http://127.0.0.1:8000/fruits/>.

OUTPUT:

Experiment-06

Develop a layout.html with a suitable header (containing navigation menu) and footer with copyright and developer information. Inherit this layout.html and create 3 additional pages: contact us, About Us and Home page of any website.

Step-01: Create project.

Step-02: Change directory and Create App. (Eg. App name: portfolioapp)

Step-03: Add the app to INSTALLED_APPS.

- Open the settings.py file in your project's directory.
- Locate the INSTALLED_APPS list and add the name of your new app to the list:

Step-04: Create a new template

- In your app's directory, create a new folder named 'templates'.
- Inside the templates folder, create another folder (portfolioapp) with the same name as your app.
- Inside the app folder, create a new files named **layout.html**, **home.html**, **about.html**, **contactus.html**
- Open **layout.html** and add the following code:

layout.html

```
<!DOCTYPE html>
<html>
<head>
    <style>
        nav{background-color: lightblue;padding: 15px;}
    </style>
    <title>
        {% block title %} {% endblock %}
    </title>
</head>
<body>
    <nav>
        <a href="{% url 'main' %}">HOME</a>
        <a href="{% url 'contact' %}">CONTACT US</a>
        <a href="{% url 'about' %}">ABOUT US</a>
    </nav>
```

```
<section>
    {% block content %} {% endblock %}
</section>
<footer>
    <hr>
    &copy; Designed and Developed by CSD Department FETC, SUK Kalburgi.
</footer>
</body>
</html>
```

home.html

```
{% extends 'portfolioapp/layout.html' %}
{% block title %} HOME Page {% endblock %}
{% block content %}
<h1>This is my home page</h1>
{% endblock %}
```

about.html

```
{% extends 'portfolioapp/layout.html' %}
{% block title %} ABOUT PAGE {% endblock %}
{% block content %}
<h1>About Us</h1>
<p> CSD Department FETC SUK KLB</p>
{% endblock %}
```

contactus.html

```
{% extends 'portfolioapp/layout.html' %}
{% block title %} Contact us {% endblock %}
{% block content %}
<h1>Contact us</h1>
<p>Name: Sharanabasappa Noola</p>
<p>Designation:Asst. Prof </p>
<p>Mobile: 8523697412</p>
<p>Email: abc@gmail.com</p>
{% endblock %}
```

Step-05: Create a view function

- Open the `views.py` file in your Django app's directory.

```
from django.shortcuts import render
def main(request):
    return render(request, 'portfolioapp/home.html')
def about(request):
    return render(request, 'portfolioapp/about.html')
def contact(request):
    return render(request, 'portfolioapp/contactus.html')
```

Step-06: Map the view function to a URL

- Open the `urls.py` file in your Django app's directory (e.g., `portfolioapp/urls.py`).
- Import the view function at the top of the file
- Add a new URL pattern to the `urlpatterns` list

```
from django.urls import path
from . import views
urlpatterns = [
    path('main/', views.main, name='main'),
    path('about/', views.about, name='about'),
    path('contact/', views.contact, name='contact'),
]
```

Step-07: Include the app's URLs in the project's URL patterns

- Open the `urls.py` file in your project's directory.
- Import the `include` function from **django.urls** and the `path` function from **django.urls**:
- Add a new URL pattern to the `urlpatterns` list

```
from django.urls import include, path [if does not exists]

path('', include(portfolioapp.urls)),
```

- This includes the URL patterns from your app's `urls.py` file.
- Include the following code:

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [
```

```
path('admin/', admin.site.urls),  
path('', include('portfolioapp.urls')),  
]
```

Step-08: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

```
python manage.py runserver
```

- Open your web browser and visit <http://127.0.0.1:8000/main/>.

OUTPUT:

